

## Lista de Exercícios

CALL — Compiler, Assembler, Linker e Loader

98800-04 – Fundamentos de Sistemas Computacionais · Prof. Iaçã Ianiski Weber

---

1. O caminho de um programa C até sua execução passa por quatro ferramentas. Para cada uma, indique o **arquivo de entrada**, o **arquivo de saída** (use como exemplo `foo.c`, `foo.s`, `foo.o` e `a.out`) e descreva, em uma frase, sua função:

a) Compiler                      b) Assembler                      c) Linker                      d) Loader

Qual dessas etapas é, na prática, realizada pelo Sistema Operacional?

2. Explique a diferença entre um **tradutor** e um **interpretador**. Por que traduzir um programa para linguagem de máquina geralmente resulta em maior desempenho do que interpretá-lo?
3. Considere o trecho de assembly abaixo (linhas numeradas):

```
1      .text
2      .globl max
3  max:  add a0, a0, a1
4      .data
5  vetor: .word 3, 9, 27
6  msg:  .string "FSC"
```

- a) Quais linhas contêm **diretivas** e quais produzem **instruções de máquina**?
- b) Explique a função de cada diretiva que aparece no trecho.
- c) Se as diretivas não geram instruções, para que elas servem no object file?

4. O compilador gerou o código abaixo:

```
1  main:  li    t0, 10
2         mv    a0, t0
3         la    a1, vetor
4         call  soma
5         ret
```

- a) Identifique todas as **pseudoinstruções** do trecho. Qual ferramenta as substitui por instruções reais?
- b) Reescreva as linhas 1, 2 e 5 usando apenas instruções reais do RV32I.
- c) Por que a pseudoinstrução `la` (linha 3) pode precisar de **duas** instruções reais?

5. O assembler processa o programa abaixo (endereços à esquerda):

```
0x00          addi t2, zero, 9
0x04  Loop:    slt  t1, zero, t2
0x08          beq  t1, zero, Exit
0x0C          addi t2, t2, -1
0x10          j    Loop
0x14  Exit:    ...
```

- a) Por que o assembler **não consegue** gerar todo o código de máquina em uma única passagem sobre esse programa? Aponte a instrução que causa o problema.
- b) Descreva o que o assembler faz em cada uma das **duas passagens**.
- c) Monte a **tabela de símbolos** do trecho (label → endereço).
- d) Calcule o deslocamento em bytes (com sinal) do `beq` e do `j`.

6. Um object file (ex: formato ELF) é composto por cinco partes.
- Cite as cinco partes e a função de cada uma.
  - No programa da questão 4, suponha que `main` foi declarada com `.globl`, que `soma` está definida em **outro arquivo** e que `vetor` está no segmento `.data` deste arquivo. Indique o que vai para a **tabela de símbolos** e o que vai para a **tabela de realocação** deste object file.
7. Para cada instrução abaixo, indique **após qual etapa** (Compiler, Assembler, Linker ou Loader) todos os 32 bits do código de máquina estão determinados. Justifique brevemente.
- `add x6, x7, x8`
  - `beq t0, zero, Loop` (rótulo `Loop` no mesmo arquivo)
  - `jal ra, printf` (`printf` em uma biblioteca)
  - `la a0, msg` (`msg` no segmento de dados estáticos)
8. O linker recebe dois object files: `file1.o` (text: 0x340 bytes; data: 0x40 bytes) e `file2.o` (text: 0x280 bytes; data: 0x20 bytes). No RV32, o primeiro segmento de texto começa em 0x10000, os segmentos de texto são concatenados na ordem dos arquivos e os segmentos de dados vêm logo após todo o texto.
- Em que endereço começa o segmento de texto de `file2.o` no executável?
  - Em que endereço começa o segmento de dados combinado?
  - A função `util` está no deslocamento 0x60 dentro do texto de `file2.o`. Qual seu endereço absoluto?
  - `file1.o` contém `jal ra, util` no deslocamento 0x120 de seu texto. Calcule o deslocamento relativo ao PC (em bytes) que o linker preenche nessa instrução.
- Curiosidade**

O nome original do linker era *Link Editor*, pois sua função era “editar os links” das instruções *jump-and-link* — exatamente o que você fez no item (d).
9. As tarefas do loader estão listadas abaixo, **fora de ordem**:
- Inicializa os registradores e aponta `sp` para a primeira posição livre da pilha.
  - Lê o cabeçalho do executável para descobrir os tamanhos dos segmentos de texto e dados.
  - Salta para a rotina de inicialização, que copia os argumentos para os registradores e define o PC.
  - Cria um espaço de endereçamento grande o suficiente para texto, dados e pilha.
  - Copia os argumentos passados ao programa para a pilha.
  - Copia instruções e dados do executável para o novo espaço de endereçamento.
- Coloque as tarefas na ordem correta de execução. O que acontece quando a função `main` retorna?
10. Uma falha de segurança foi corrigida na biblioteca `libc`. Considere dois programas já instalados: **A**, linkado **estaticamente**, e **B**, linkado **dinamicamente**.
- Qual dos dois recebe a correção sem precisar ser recompilado? Explique.
  - Cite uma vantagem da abordagem estática e uma da dinâmica.
  - Por que bibliotecas dinâmicas economizam **memória** quando dois programas que as utilizam executam ao mesmo tempo?